- *When you're converting an existing MVC application to ASP.NET Core*—If you already have a legacy ASP.NET application that you're converting to ASP.NET Core or an app using an early version of ASP.NET Core that you're updating, you're likely using MVC controllers. It's probably not worth converting your existing MVC controllers to Razor Pages in this case. It makes more sense to keep your existing code and consider whether to do *new* development in the application with Razor Pages.
- *When you're doing a lot of partial page updates*—It's possible to use JavaScript in an MVC application to avoid doing full page navigations by updating only part of the page at a time. This approach, halfway between fully server-side rendered and a client-side application, may be easier to achieve with MVC controllers than Razor Pages. On the other hand, you can easily mix Razor Pages and MVC controllers, using Razor Pages where appropriate and MVC controllers for the partial view results.

When not to use Razor Pages or MVC controllers

Typically, you'll use either Razor Pages or MVC controllers to write most of the UI logic for an app. You'll use it to define the APIs and pages in your application and to define how they interface with your business logic. Razor Pages and MVC provide an extensive framework and include a great deal of functionality to help build your apps quickly and efficiently. But they're not suited to every app.

Providing so much functionality necessarily comes with a certain degree of performance overhead. For typical line-of-business apps, the productivity gains from using MVC or Razor Pages often outweighs any performance effect. But if you're building a JSON API you will